

KEYCLOAK & S3 PRIMER — SHEET 01

Keycloak & S3, the basics

Before you dig into realms, clients, and presigned URLs for the report assignment — here's the floor everything else stands on: what these two services are, and the one problem each solves.

SHEET 02 - WHY NOT ROLL YOUR OWN AUTH

Passwords are a liability you don't want to own

Hashing correctly, sessions, password resets, MFA, breach monitoring — all of that has to exist before your app does anything user-facing. Keycloak's job is to be the one place that owns identity, so your app never touches a password directly.



Four words you'll see everywhere

01

Realm

An isolated tenant. Users, clients, and roles in one realm are invisible to another.

02

Client

An application registered to use the realm — this is "your app," from Keycloak's point of view.

03

User

A person who can log in. Belongs to a realm, can belong to groups.

04

Group / Role

How you say what a user can do. A group is a bucket of users; a role is a named permission.

SHEET 04 - THE TOKEN FLOW

Log in once, prove it on every request

user submits credentials

→

Keycloak issues a signed JWT

→

app verifies it offline

The app checks the token's signature against Keycloak's public key and reads the claims inside (who, which groups) — without calling Keycloak again on every request. That verification step is what you'll find inside the backend's auth dependency.

Blob storage is a different job than a database

A database is built for structured, queryable rows — not for storing a 20MB image or video efficiently. Object storage (S3, or MinIO locally) exists specifically to store and serve arbitrary files cheaply, at scale.

Database	stores rows, relationships — the things you query and join
S3 / MinIO	stores files (objects), referenced by a URL your database points to

Three words, one mental model

01

Bucket

A namespace you create once — think top-level folder.

02

Key

The path/name of a file inside a bucket, e.g. uploads/avatars/42.png.

03

Object

The actual file plus a little metadata, addressed by bucket + key together.

A temporary, signed door to one object

Your storage credentials must stay secret on the backend. A presigned URL is a time-limited link the backend generates that grants one specific action (GET or PUT) on one specific object — without ever handing out the real keys.

backend generates a 5-minute link

→

browser uploads/downloads directly to S3

- This is why a large file upload doesn't have to pass through — or overload — your API server.

SHEET 08 — WHERE IT CONNECTS

The shape of a typical request

Frontend	sends the Keycloak JWT on every API call
Backend	verifies the JWT, then asks S3 for a presigned URL
Frontend	uploads/downloads straight to S3 with that URL
Database	only ever stores the object's key/URL — never the file bytes

SHEET 09 — NOW GO DEEPER

This is the floor, not the ceiling.

This primer gives you the shape. The report assignment still wants you to dig into the specifics inside this codebase — exactly how tokens get refreshed, what the Keycloak adapter does, the real difference between a confidential and a public client, where a file upload is actually handled in the code. That part only reading the real code will answer.

REFERENCE ONLY

→ THE REST IS YOUR RESEARCH: `REPORT_ASSIGNMENT.MD`